Tutti

Peer-to-peer tutoring platform

Deliverable 1

Requirements and modelling

Course	Software Engineering 1 · Winter semester 2026/27
Team	Nikita · Nicolle · Anastasiia · Stefan · Yanislav
Contents	Stakeholders, user stories, story maps, requirements analysis, use-case model, domain model, sequence diagrams
Date	October 2026

Contents

1 Requirements elicitation 3
1.1 Stakeholders (Nicolle) 3
1.2 User stories 5
1.3 User story mapping (Anastasiia) 7
1.4 Requirements analysis 8
2 Use-case model (Nikita) 13
2.1 Actors 14
2.2 Use-case descriptions 14
3 Domain model (Stefan) 22
4 Behavioural model (Yanislav) 23
4.1 Part 1 — Find a tutor (UC02) 23
4.2 Part 2 — Request a session (UC03) 24
4.3 Part 3 — Tutor responds (UC09) 25

1 Requirements elicitation

This section identifies who has a stake in Tutti and what they need, turns those needs into user stories and story maps, and derives the functional and non-functional requirements.

1.1 Stakeholders (Nicolle)

Each stakeholder has a category (how it relates to the system) and a role in requirements engineering. Goals, expectations and concerns are written from the stakeholder's own point of view.

Student	PRIMARY
Role	Requirement provider, validator
Description	Enrolled student who looks for help and requests tutoring sessions.
Goal	Quickly find a tutor approved for my subject, by faculty, program and subject, at a time that fits my timetable.
Expectation	Choose online or onsite, explain the purpose and what I need, and get a quick answer from the tutor.
Concern	Being sure the tutor is competent, being able to cancel when my plans change, and knowing whom to contact if a session goes wrong.

Tutor	PRIMARY
Role	Requirement provider, validator
Description	Student approved by the coordinator to tutor one or more subjects.
Goal	Become a tutor through a simple application and offer the subjects I am good at.
Expectation	Decide when and how I teach by setting my own free hours (online or onsite), accepting or declining requests, and knowing in advance what the student needs.
Concern	Students who cancel late or don't show up, and my grades or personal data being visible to other students.

Coordinator (one per faculty)	PRIMARY
Role	Decision-maker, validator, domain expert
Description	Staff member who approves tutors, supervises tutoring in the faculty and resolves issues.
Goal	Make sure only qualified students (at least 15/20 in the subject) become tutors.
Expectation	A clear overview of the faculty's tutors: schedules, number of sessions, ratings, and users who repeatedly cancel late or don't show up.
Concern	Being able to act quickly when something goes wrong: all issue reports in one place, and the power to cancel sessions that break the rules or suspend users.

Lecturers	SECONDARY
Role	Domain expert, requirement provider
Description	Teach the courses in which students receive tutoring; they don't use the platform themselves.
Goal	Students who struggle in my course get extra help.
Expectation	Tutors explain the course material correctly and at the right level.
Concern	Academic integrity, meaning tutors must not do graded work for students.

Sponsor (non-university)	SECONDARY
Role	Requirement provider
Description	Organisation that funds the platform from outside the university and receives reports on its use.
Goal	Visibility, through an acknowledgement of the sponsor on the platform.
Expectation	Evidence of impact, through periodic reports on active users, sessions held and feedback scores.
Concern	Reports that are reliable and comparable over time, without exposing personal data.

Data Protection Officer (university)	EXTERNAL
Role	Constraint source, validator
Description	Ensures the platform complies with the GDPR.
Goal	Personal data (grades, schedules, contact details) is collected only when it is needed.
Expectation	Access depends on the role: grades are visible only to the coordinator, and each coordinator sees only their own faculty.
Concern	Data leaks, and data kept longer than necessary.

Development team (us)	HIDDEN
Role	Maintainer/operator, decision-maker on technical feasibility
Description	Builds, deploys and maintains the platform.
Goal	Deliver a working platform within the semester and its three deliverables.
Expectation	Clear, prioritized requirements that we refine with the stakeholders at every sprint.
Concern	A modular, testable structure that is easy to maintain and extend, with secure handling of personal data.

1.2 User stories

Stories are labelled S (Student), T (Tutor) and C (Coordinator); the Source column of the functional requirements refers to these labels.

Student

ID	User story
S1	As a student, I want to find tutors by faculty, program and subject so that I get help from someone approved for my course.
S2	As a student, I want to request a session by choosing a free time, a duration, online or onsite, a purpose and a description so that the tutor knows what I need before accepting.
S3	As a student, I want to be notified by e-mail when my request is accepted, declined or expires so that I know whether the session will take place.
S4	As a student, I want to cancel a session with a reason so that the tutor's time is freed for others.
S5	As a student, I want to rate the tutor after a session so that other students and the coordinator can judge tutoring quality.

Tutor

ID	User story
T1	As a student who wants to tutor, I want to apply with a description, my subjects and my grades so that the coordinator of my faculty can approve me.
T2	As a tutor, I want to add my free time windows and mark each one as online, onsite or both so that students only request times that suit me.
T3	As a tutor, I want to accept or decline session requests, giving a reason when I decline, so that I only commit to sessions I can give.
T4	As a tutor, I want to cancel an accepted session with a reason so that the student is informed and can request a new one.
T5	As a tutor, I want to see how many sessions and hours I have taught so that I can track my tutoring activity.

Coordinator

ID	User story
C1	As a coordinator, I want to approve or reject tutor applications from my faculty so that only students with at least 15/20 in a subject tutor it.
C2	As a coordinator, I want to see each tutor's schedule, number of sessions and ratings so that I can monitor tutoring in my faculty.
C3	As a coordinator, I want to receive all issue reports in one inbox so that I can resolve problems quickly.
C4	As a coordinator, I want to be alerted when a student or tutor reaches 2 late cancellations or no-shows so that I can take action.

ID	User story
C5	As a coordinator, I want to cancel sessions that break the policy so that the platform rules are respected.

1.3 User story mapping (Anastasiia)

Each map follows one user journey from left to right. Cards above the MVP line form the first release; cards between the MVP and Release 1 lines come next.

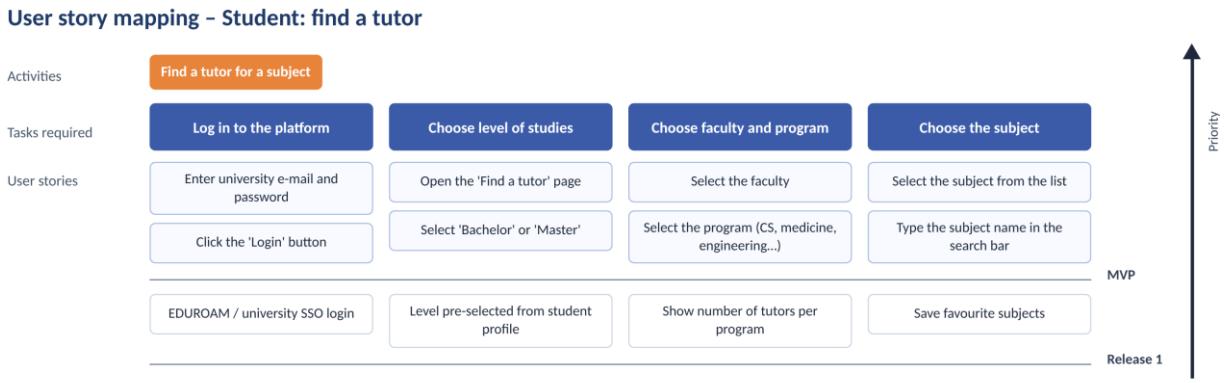


Figure 1. *User story map – Student: find a tutor*

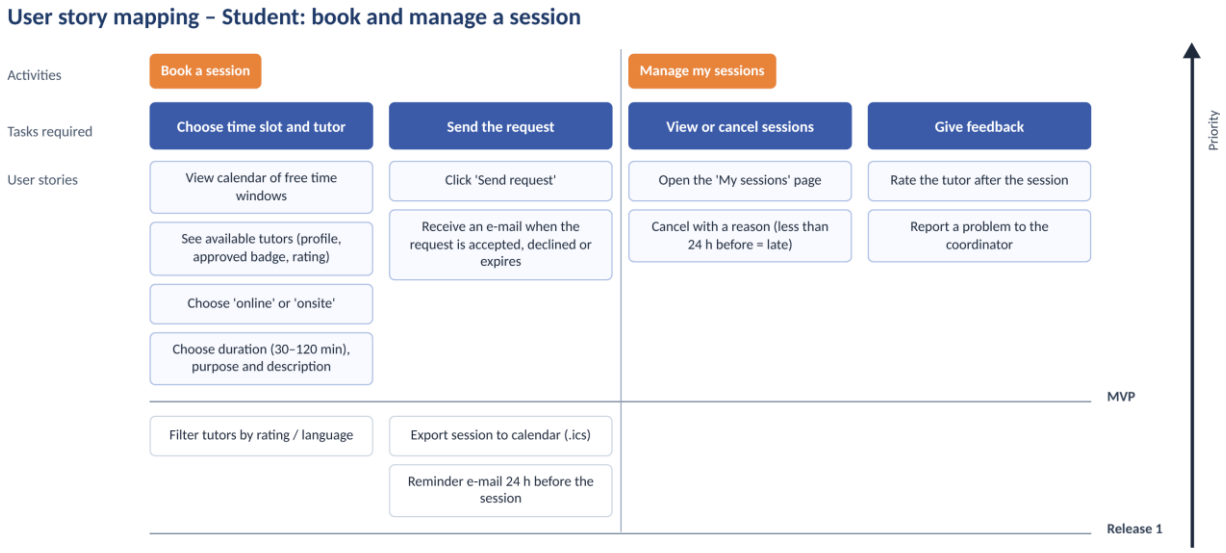


Figure 2. *User story map – Student: book and manage a session*



Figure 3. *User story map – Tutor: become a tutor and offer subjects*

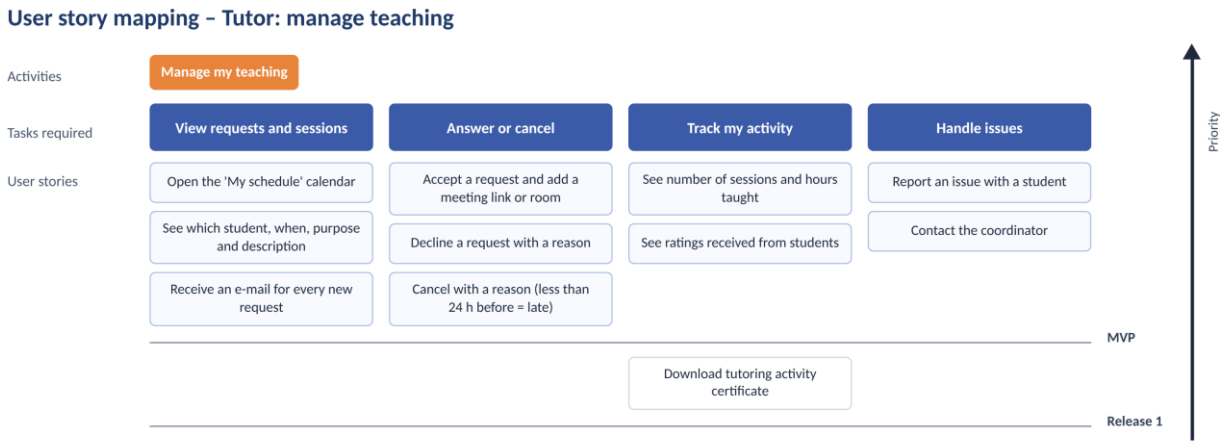


Figure 4. *User story map – Tutor: manage teaching*

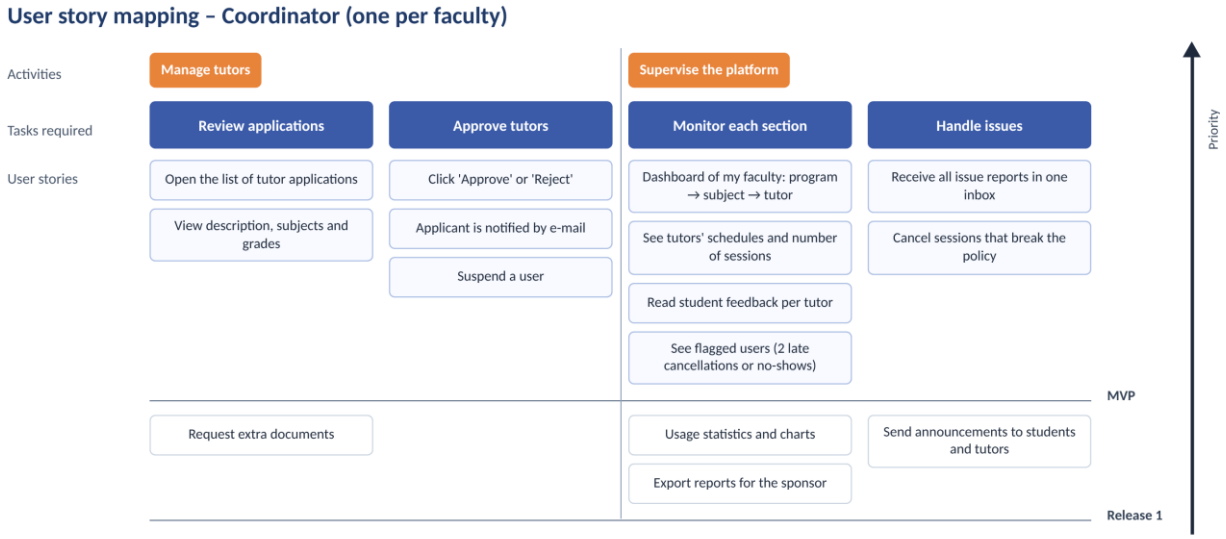


Figure 5. *User story map – Coordinator (one per faculty)*

1.4 Requirements analysis

Requirements are derived from the stakeholder needs and checked against the user stories and use cases. Priorities follow the MVP and Release 1 lines of the story maps (MoSCoW).

Functional requirements

ID	Requirement	Priority	Source
FR1	The system shall let users log in with their university e-mail and password and open the dashboard of their role (Student, Tutor, Coordinator).	**Must**	All roles · UC01
FR2	The system shall let students search tutors by level, faculty, program and subject, showing only approved, active tutors with their rating and free time windows.	**Must**	S1 · UC02
FR3	The system shall let students request a session inside a tutor's free window by choosing a start time, a duration (30, 60, 90 or 120 min), a mode, a purpose and a description.	**Must**	S2 · UC03

ID	Requirement	Priority	Source
FR4	The system shall accept only 30 min for the purpose "Introduction" and at least 60 min for every other purpose.	**Must**	S2 · UC03
FR5	The system shall block the requested time so that no other request can overlap it.	**Must**	S2 · UC03
FR6	The system shall expire a request that the tutor has not answered after 48 h or 24 h before its start, whichever comes first, and free the time.	**Must**	S3 · UC03
FR7	The system shall notify users by e-mail, sent to their university e-mail address, when a request is sent, accepted, declined, expired or cancelled, and when a tutor application is decided.	**Must**	S3 · UC15
FR8	The system shall let the student or the tutor cancel a session with a mandatory reason and mark the cancellation as late if it is less than 24 h before the start.	**Must**	S4, T4 · UC04
FR9	The system shall let a student rate a completed session once, from 1 to 5, with an optional comment.	**Must**	S5 · UC05
FR10	The system shall let students and tutors report an issue, optionally linked to a session, to the coordinator of the faculty.	**Must**	C3 · UC06
FR11	The system shall let students apply as tutors with a description, subjects and grades, accept only subjects graded at least 15/20, and send the application to the coordinator of their faculty.	**Must**	T1 · UC07
FR12	The system shall let tutors create, edit and delete availability windows marked online, onsite or both, rejecting overlaps and refusing to delete windows that contain sessions.	**Must**	T2 · UC08
FR13	The system shall let tutors accept a request (adding a meeting link or room) or decline it with a reason.	**Must**	T3 · UC09
FR14	The system shall show tutors their upcoming sessions, number of completed sessions, hours taught and ratings received.	**Must**	T5 · UC10
FR15	The system shall let coordinators approve or reject (with a reason) tutor applications from their faculty; approval creates or extends the student's tutor profile.	**Must**	C1 · UC11
FR16	The system shall give coordinators a dashboard of their faculty's tutors with schedules, number of sessions, ratings and feedback.	**Must**	C2 · UC12
FR17	The system shall collect all issue reports of a faculty in the coordinator's inbox and let the coordinator close them with a resolution note.	**Must**	C3 · UC13
FR18	The system shall count each user's late cancellations and confirmed no-shows per semester and flag the user to the coordinator when the count reaches 2.	**Must**	C4 · UC16

ID	Requirement	Priority	Source
FR19	The system shall let coordinators cancel sessions that break the policy and suspend users.	**Must**	C5 · UC04, UC14
FR20	The system shall send a reminder 24 h before each accepted session.	**Should**	Story map (Release 1)
FR21	The system shall let coordinators export a usage report for the sponsor (active users, sessions held, average feedback).	**Should**	Sponsor needs (Release 1)
FR22	The system shall display the sponsor's acknowledgement on the platform.	**Should**	Sponsor needs
FR23	The system shall support login through the university SSO.	**Could**	Story map (Release 1)
FR24	The system shall let users export a session to their calendar (.ics).	**Could**	Story map (Release 1)
FR25	Payments between users.	**Won't**	Decision: platform is free
FR26	Tutors proposing an alternative time for a request.	**Won't**	Decision: tutor declines, student sends a new request
FR27	In-app notifications.	**Won't**	Decision: all communication is by e-mail

Non-functional requirements

ID	Category	Requirement
NFR1	Security	Every page except login requires authentication; each role sees only the functions of that role.
NFR2	Security	A coordinator can see and change only the data of their own faculty.
NFR3	Security	Passwords are stored hashed and all traffic uses HTTPS.
NFR4	Privacy (GDPR)	Tutors' grades are visible only to the coordinator of their faculty; students see an "approved for subject" badge instead.
NFR5	Privacy (GDPR)	Sponsor reports contain only aggregated, anonymous figures.
NFR6	Usability	A student can send a session request in at most 3 steps after opening a tutor's profile.
NFR7	Usability	The interface works on current desktop and mobile browsers.
NFR8	Performance	Search results and request confirmations appear within 2 s with 200 concurrent users.
NFR9	Reliability	Every notification e-mail is recorded with its status (pending, sent or failed) and the time it was sent.

ID	Category	Requirement
NFR10	Availability	The platform is available 99% of the time during the semester.
NFR11	Maintainability	The code is split into UI, controller, service and data layers; business rules (durations, expiry, strikes) are covered by automated tests; all code is versioned in Git and reviewed before merging.
NFR12	Traceability	Every status change of a session or application is logged with time and user.

Stakeholder conflicts

Conflict	Stakeholders	Resolution
Students want to cancel late without consequences; tutors want enough notice.	Student vs Tutor	Cancellation is always possible, but less than 24 h before the session it is recorded as late; 2 late cancellations or no-shows per semester flag the user.
Students want to see tutors' grades to judge competence; tutors' grades are personal data.	Student vs Tutor, Data Protection Officer	Only the coordinator sees grades; students see an "approved" badge and ratings.
Coordinators want full access to student and tutor data; data protection requires minimal access.	Coordinator vs Data Protection Officer	Each coordinator sees only their own faculty.
The sponsor wants usage figures and visibility; students expect privacy.	Sponsor vs Student	Reports contain only aggregated figures; branding is an acknowledgement, no ads.
Tutors would like to propose other times; the team must keep the scope realistic.	Tutor vs Development team	No counter-proposals: the tutor declines with a reason and the student sends a new request.
Students want help with homework; lecturers require that graded work is the student's own.	Student vs Lecturers	Homework sessions explain methods only; the coordinator can cancel sessions that break this rule.

Assumptions

- The platform is free; no payments are handled.
- All users have a university e-mail address; tutors are enrolled students.
- Sessions are one-to-one: one tutor and one student.
- There is one coordinator per faculty; a tutor application goes to the coordinator of the applicant's faculty.
- Session start times follow a 30-minute grid.
- The count of late cancellations and no-shows resets every semester.
- In the MVP, applicants declare their grades themselves; transcript upload comes in Release 1.
- The sponsor does not use the system; it receives reports exported by the coordinators.
- All communication with users is by e-mail to their university address; there are no in-app notifications.

Open questions

- How long is personal data kept after a student graduates or leaves?
- What action follows a flag (warning, temporary suspension)? For now the coordinator decides case by case.
- Until transcript upload exists, how does the coordinator verify the grades that applicants declare?

2 Use-case model (Nikita)

The use-case model formalises the functional requirements: who interacts with Tutti (actors), which goals the system supports (use cases), and how the use cases relate through «include», «extend» and generalization. A Tutor is a specialization of Student, so tutors can also book sessions.

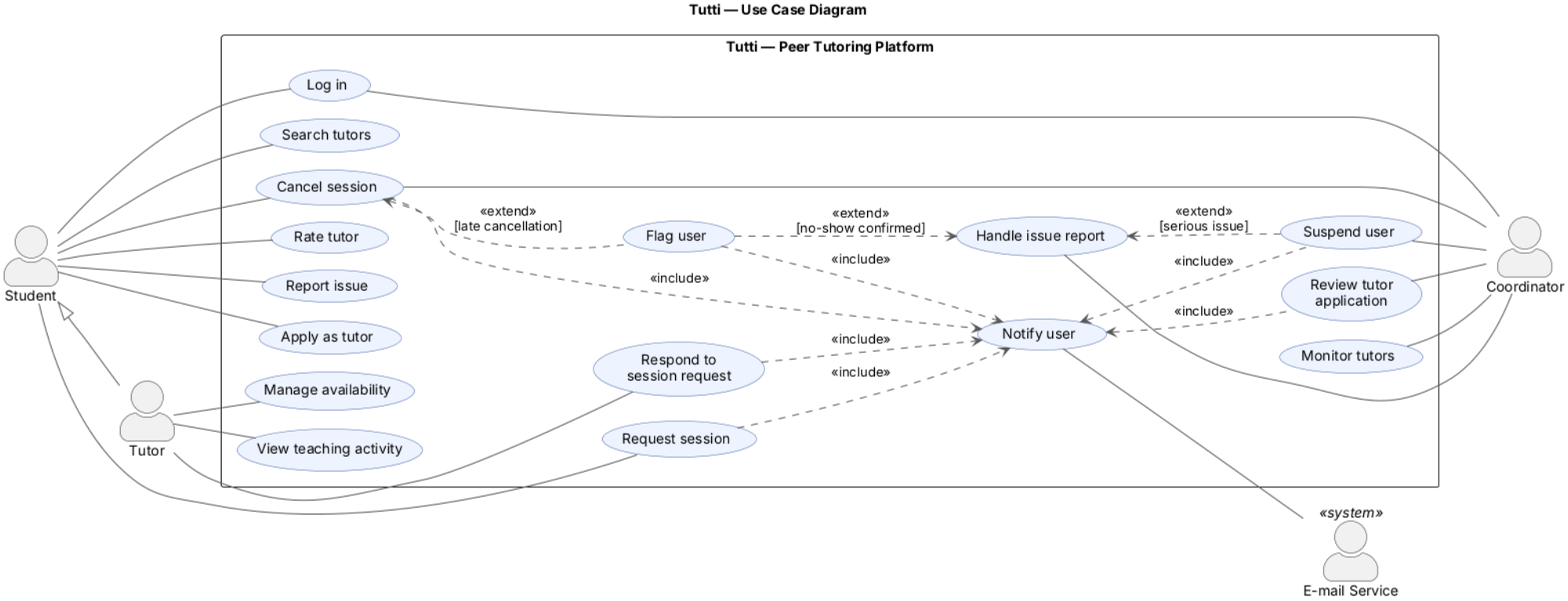


Figure 6. *Use-case diagram of Tutti*

2.1 Actors

Actor	Type	Description
Student	Primary	Any enrolled student. Searches tutors, requests and cancels sessions, rates tutors, reports issues, applies to become a tutor.
Tutor	Primary	A student approved for one or more subjects. Specializes Student, so a tutor can also book sessions as a student.
Coordinator	Primary	One per faculty. Approves tutors, monitors tutoring, handles issues and suspends users.
E-mail Service	Secondary (system)	External service that delivers e-mail notifications.

2.2 Use-case descriptions

Each description gives the actors, goal, preconditions, main success scenario, alternative flows (extensions, numbered after the step where they branch off) and postconditions.

UC01 Log in	
Actors	Student (primary), Coordinator (primary)
Goal	Access the platform with the right role.
Preconditions	The user has a university account.
Main success scenario	1. User enters university e-mail and password. 2. System verifies the credentials. 3. System opens the dashboard for the user's role (Student, Tutor or Coordinator).
Extensions	**2a. Wrong credentials:** System shows an error and the user retries. **2b. Account suspended:** System denies access and shows the reason.
Postconditions	User is authenticated, or access is denied.

UC02 Search tutors	
Actors	Student (primary)
Goal	Find an approved tutor for a subject.
Preconditions	Student is authenticated.
Main success scenario	1. Student selects level (Bachelor/Master), faculty, program and subject, or types the subject name. 2. System lists active tutors approved for that subject, with their "approved" badge, average rating and free time windows. 3. Student opens a tutor's profile.
Extensions	**2a. No tutor for the subject:** System shows "No tutors available for this subject yet".
Postconditions	Student sees the matching tutors (or none). Tutors' grades are never shown.

UC03 Request session	
Actors	Student (primary), E-mail Service (secondary)
Goal	Ask a tutor for a session at a specific time.
Preconditions	Student is authenticated and has opened a tutor's profile (UC02).
Main success scenario	1. Student picks a start time inside one of the tutor's free windows. 2. Student chooses the duration (30, 60, 90 or 120 min), the mode (online/onsite, if the window allows both), the purpose (Introduction, Homework, Exam preparation, …) and writes a description. 3. System checks the rules: Introduction = 30 min; any other purpose ≥ 60 min; the time is still free. 4. System creates the request with status *Requested* and blocks that time. 5. System includes **Notify user** to inform the tutor. 6. System shows "Request sent".
Extensions	**3a. Purpose and duration don't match:** System shows the allowed durations and the student corrects them. **3b. Time no longer free:** System shows an error and refreshes the free windows. **4a. Tutor doesn't answer in time:** after 48 h, or 24 h before the session (whichever comes first), the Scheduler marks the request *Expired*, the time is freed and the student is informed by e-mail.
Postconditions	A pending request exists and the time is blocked, or nothing is created.

UC04 Cancel session	
Actors	Student or Tutor (primary), Coordinator (primary), E-mail Service (secondary)
Goal	Cancel a requested or accepted session.
Preconditions	User is authenticated and is the session's student, its tutor, or the coordinator.
Main success scenario	1. User opens the session and selects "Cancel". 2. User enters a reason (mandatory). 3. System sets the status to *Cancelled*, records who cancelled and frees the time. 4. System includes **Notify user** to inform the other party.
Extensions	**1a. Coordinator cancels for a policy violation:** both student and tutor are notified with the reason. **3a. Less than 24 h before the session (student or tutor):** the cancellation is recorded as late and **Flag user** extends the use case.
Postconditions	Session is cancelled and both parties know why. To meet another time, the student sends a new request.

UC05 Rate tutor	
Actors	Student (primary)
Goal	Give feedback on a completed session.
Preconditions	Student is authenticated; the session is *Completed* and not yet rated.
Main success scenario	1. Student opens the completed session. 2. Student gives a rating (1–5) and an optional comment. 3. System saves the feedback and updates the tutor's average rating.
Extensions	**1a. Session already rated:** System shows the existing rating; no second rating is possible.
Postconditions	Feedback is stored and visible to the tutor and the coordinator.

UC06 Report issue	
Actors	Student or Tutor (primary)
Goal	Tell the coordinator about a problem.
Preconditions	User is authenticated.
Main success scenario	1. User selects "Report a problem", optionally from a specific session. 2. User describes the issue (e.g. no-show, inappropriate behaviour). 3. System creates an issue report with status *Open* and places it in the inbox of the coordinator responsible for that faculty. 4. System confirms the report was sent.
Extensions	**2a. Description empty:** System asks the user to fill it in.
Postconditions	An open issue report exists in the coordinator's inbox.

UC07 Apply as tutor	
Actors	Student (primary)
Goal	Become a tutor for one or more subjects.
Preconditions	Student is authenticated and has no pending application.
Main success scenario	1. Student opens "Become a tutor". 2. Student writes a description and selects subjects, entering the grade obtained in each. 3. System checks that each grade is at least 15/20. 4. System saves the application with status *Pending* and sends it to the coordinator of the student's faculty. 5. System shows "Application submitted".
Extensions	**1a. Pending application already exists:** System shows its status instead of a new form. **3a. A grade is below 15/20:** System marks that subject as not eligible; the student removes it or cancels.
Postconditions	A pending application waits for review.

UC08 Manage availability	
Actors	Tutor (primary)
Goal	Publish the times when students can request sessions.
Preconditions	Tutor is authenticated and approved.
Main success scenario	1. Tutor opens the availability calendar. 2. Tutor adds, edits or deletes a time window and marks it online, onsite or both. 3. System checks that the window does not overlap another one. 4. System saves the change and shows the window to students.
Extensions	**2a. Window contains requested or accepted sessions:** System refuses to delete or shrink it until those sessions are cancelled (UC04). **3a. Overlap:** System rejects the window and highlights the conflict.
Postconditions	The tutor's free windows are up to date.

UC09 Respond to session request	
Actors	Tutor (primary), E-mail Service (secondary)
Goal	Accept or decline a student's request.
Preconditions	Tutor is authenticated; a request with status *Requested* exists.
Main success scenario	1. Tutor opens the request (from the e-mail link or the list of requests) and reads the student, date, duration, mode, purpose and description. 2. Tutor accepts and adds a meeting link (online) or a room (onsite). 3. System checks that the request is still *Requested* and sets the status to *Accepted*. 4. System includes **Notify user** to inform the student by e-mail.
Extensions	**2a. Tutor declines:** Tutor enters a reason; System sets the status to *Declined*, frees the time and informs the student by e-mail. **3a. Request no longer pending (expired or cancelled meanwhile):** System shows "This request is no longer pending"; nothing changes.
Postconditions	Request is accepted with meeting details, or declined with a reason.

UC10 View teaching activity	
Actors	Tutor (primary)
Goal	Track own tutoring activity.
Preconditions	Tutor is authenticated.
Main success scenario	1. Tutor opens "My activity". 2. System shows upcoming sessions, number of completed sessions, hours taught and ratings received.
Extensions	**2a. No sessions yet:** System shows empty counters.
Postconditions	— (read only)

UC11 Review tutor application	
Actors	Coordinator (primary), E-mail Service (secondary)
Goal	Approve or reject a student who wants to tutor.
Preconditions	Coordinator is authenticated; a pending application from their faculty exists.
Main success scenario	1. Coordinator opens the list of pending applications. 2. Coordinator reads the description, subjects and grades. 3. Coordinator approves the application. 4. System sets it to *Approved*, creates (or extends) the student's tutor profile with the approved subjects. 5. System includes **Notify user** to inform the student.
Extensions	**3a. Coordinator rejects:** Coordinator enters a reason; System sets the application to *Rejected* and notifies the student.
Postconditions	The student is a tutor for the approved subjects, or knows why not.

UC12 Monitor tutors	
Actors	Coordinator (primary)
Goal	Oversee tutoring in the faculty.
Preconditions	Coordinator is authenticated.
Main success scenario	1. Coordinator opens the dashboard and drills down: program → subject → tutor. 2. System shows each tutor's schedule, number of sessions, ratings and student feedback. 3. System lists flagged users (2 late cancellations or no-shows this semester).
Extensions	**2a. Coordinator spots a problem:** Coordinator continues with UC14 (**Suspend user**) or UC04 (**Cancel session**).
Postconditions	— (read only)

UC13 Handle issue report	
Actors	Coordinator (primary)
Goal	Resolve a reported problem.
Preconditions	Coordinator is authenticated; an open issue report exists in their inbox.
Main success scenario	1. Coordinator opens the issue from the inbox. 2. Coordinator reviews the description and the related session. 3. Coordinator contacts the people involved if needed and decides. 4. Coordinator closes the issue with a resolution note.
Extensions	**3a. No-show confirmed: Flag user** extends the use case for the absent person. **3b. Serious issue: Suspend user** extends the use case.
Postconditions	The issue is *Resolved* with a note.

UC14 Suspend user	
Actors	Coordinator (primary)
Goal	Stop a student or tutor from using the platform.
Preconditions	Coordinator is authenticated.
Main success scenario	1. Coordinator selects the user and "Suspend". 2. Coordinator enters a reason. 3. System suspends the account, hides the tutor profile (if any) and cancels the user's future sessions. 4. System includes **Notify user** to inform the suspended user and the other parties of the cancelled sessions by e-mail.
Extensions	**3a. User has no future sessions:** only the account is suspended.
Postconditions	The user cannot log in; affected sessions are cancelled.

UC15 Notify user «included»	
Actors	E-mail Service (secondary)
Goal	Inform a user by e-mail that something changed. All communication is by e-mail; there are no in-app notifications.
Preconditions	Triggered by UC03, UC04, UC09, UC11, UC14 or UC16, or by request expiry.
Main success scenario	1. System records the notification (type, subject, body) with e-mail status *Pending*. 2. System sends it through the E-mail Service to the recipient's university e-mail address (User.email). 3. E-mail Service accepts the message; System sets the e-mail status to *Sent* and records the time.
Extensions	**3a. E-mail Service rejects the message:** System sets the e-mail status to *Failed*.
Postconditions	The recipient has received the e-mail, and the notification is recorded with its e-mail status.

UC16 Flag user «extend»	
Actors	— (system behaviour)
Goal	Detect students and tutors who repeatedly cancel late or don't show up.
Preconditions	Extends UC04 (late cancellation) or UC13 (confirmed no-show).
Main success scenario	1. System adds one to the user's count of late cancellations and no-shows for the current semester. 2. The count reaches 2. 3. System flags the user. 4. System includes **Notify user** to inform the coordinator of the user's faculty by e-mail.
Extensions	**2a. Count is below 2:** the event is only recorded.
Postconditions	The event is recorded; at 2, the user is flagged for the coordinator.

3 Domain model (Stefan)

The class diagram captures the concepts Tutti must remember, their attributes and relationships. Derived attributes are marked with "/" and their formulas, like the business rules, are given in the notes.

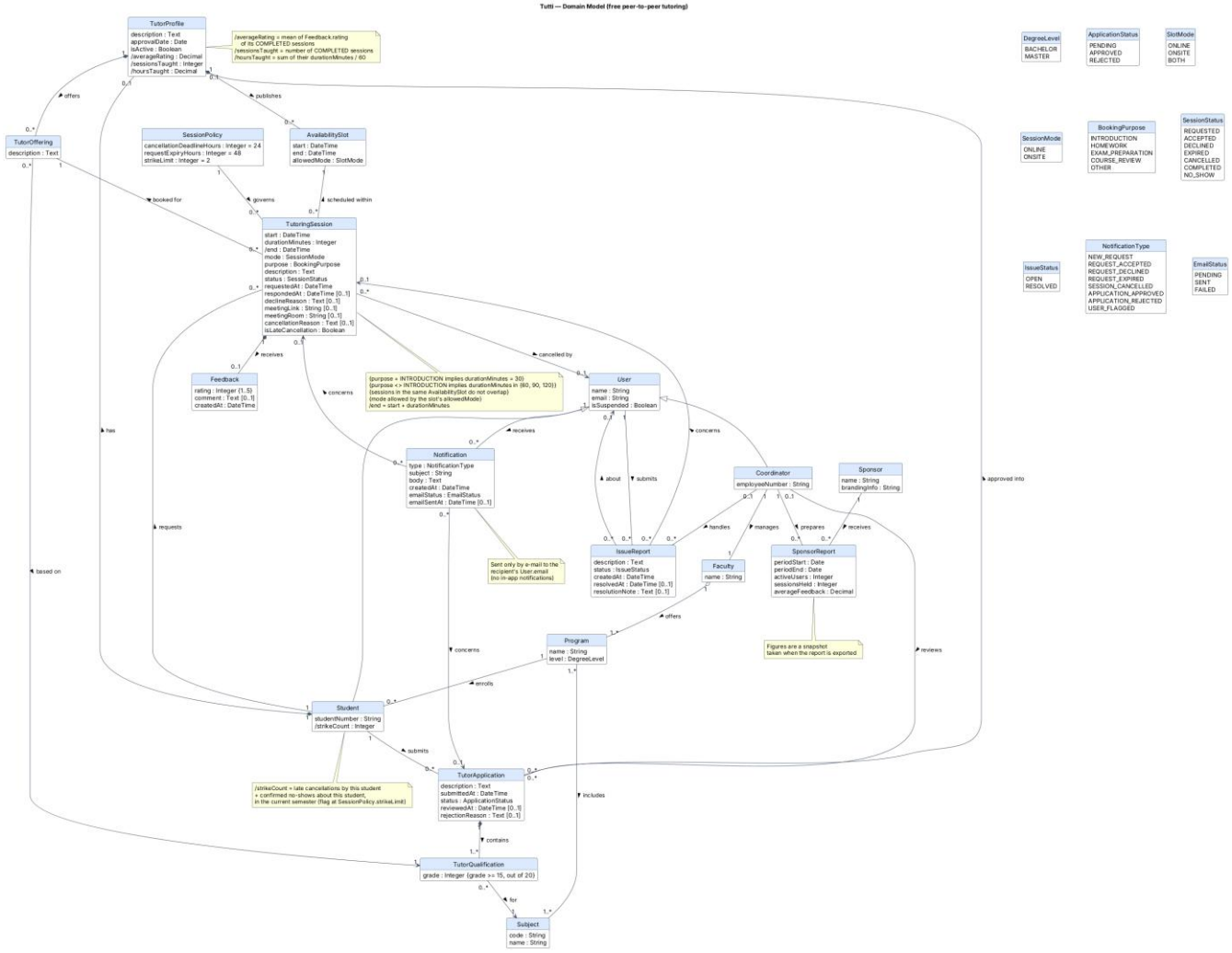


Figure 7. *UML class diagram (domain model)*

4 Behavioural model (Yanislav)

The sequence diagram shows how Tutti realises the core booking journey across three use cases: finding a tutor (UC02), requesting a session (UC03) and the tutor's response (UC09). It is split into three parts; message numbers continue from one part to the next. All notifications are e-mails sent to the user's university address.

4.1 Part 1 — Find a tutor (UC02)

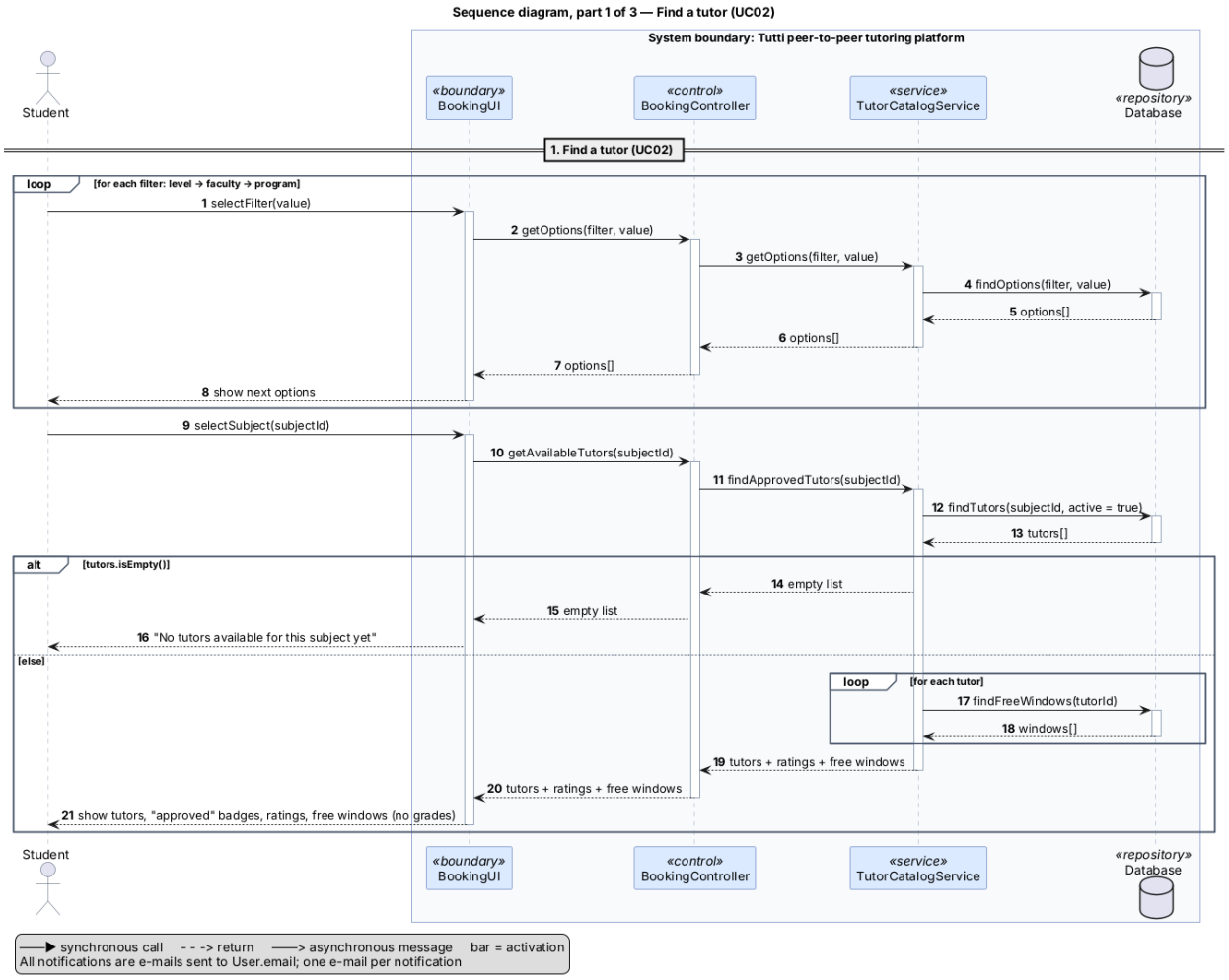


Figure 8. *Sequence diagram, part 1: find a tutor (messages 1–21)*

4.2 Part 2 — Request a session (UC03)

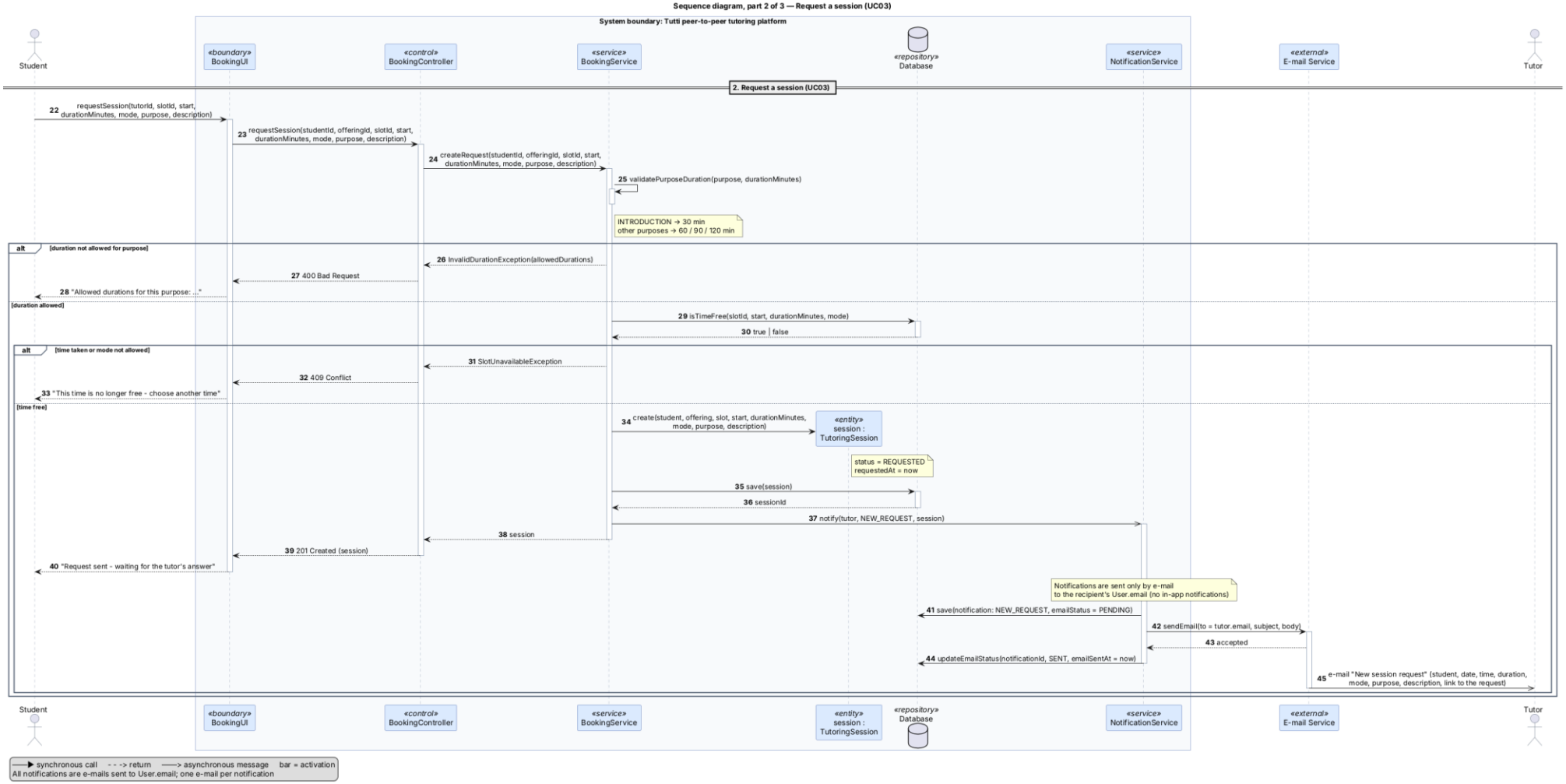


Figure 9. *Sequence diagram, part 2: request a session (messages 22–45)*

4.3 Part 3 — Tutor responds (UC09)

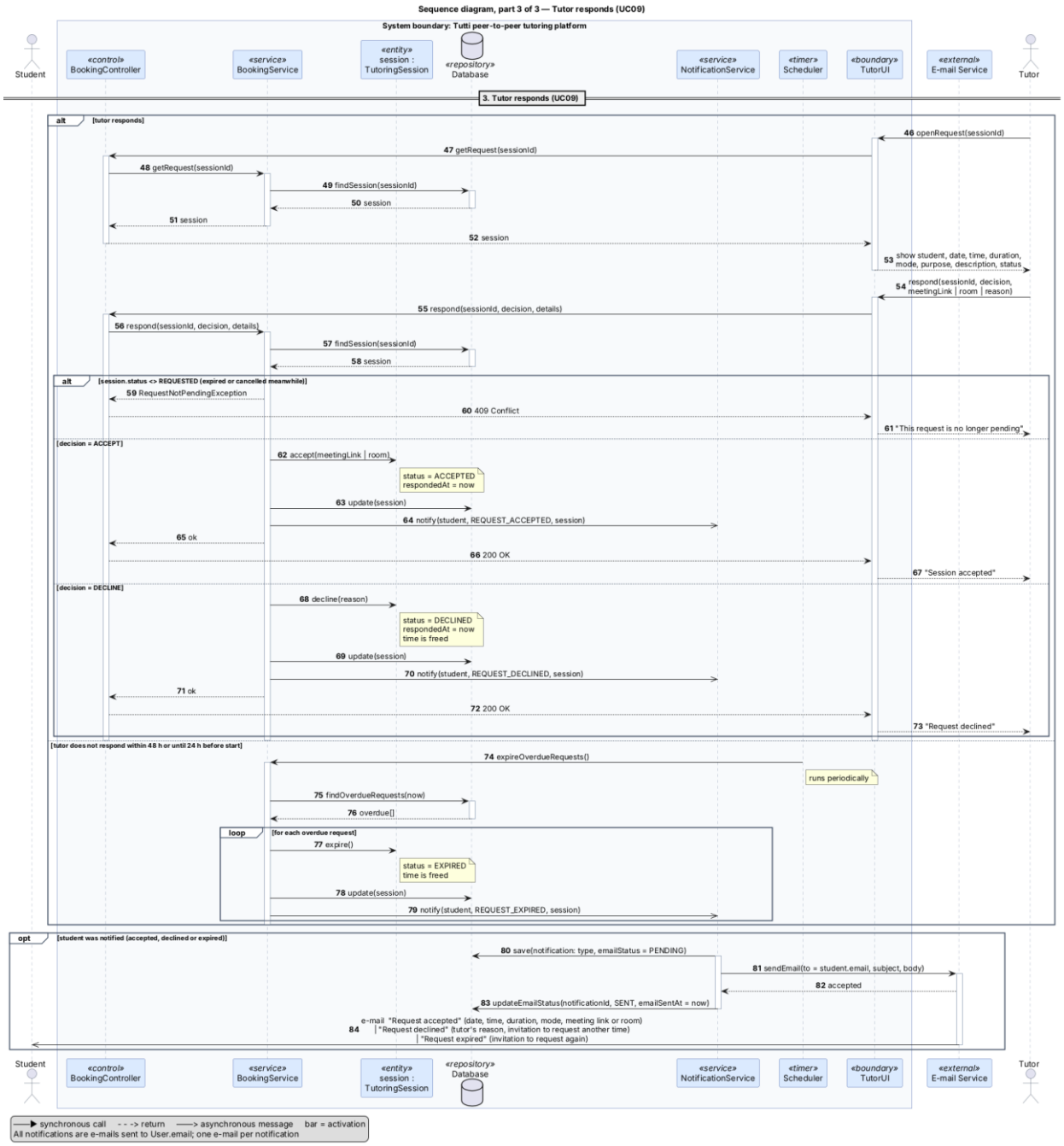


Figure 10. *Sequence diagram, part 3: tutor responds (messages 46–84)*